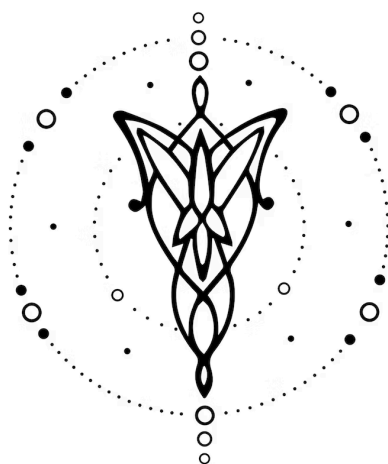




Stujfy

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

3 de Septiembre de 2026

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	3
5. Casos de prueba	4
6. Ejemplos	5

1. Equipo

Nombre	Apellido	Legajo	E-mail
Jesus Gabriel	Diaz	64475	jbastidasdiaz@itba.edu.ar
Francesco	Vega Scarabino	65199	fvegascarabino@itba.edu.ar

2. Repositorio

La solución será versionada en: https://github.com/jbdiaz12/TP_TLA_2026C2

3. Dominio

El lenguaje propuesto permite describir planes de aprendizaje y resolver automáticamente su distribución en el tiempo. Se declaran los objetivos con su fecha límite, los temas que los componen y la disponibilidad horaria real de quien estudia; a partir de esa descripción, el compilador construye el cronograma de sesiones. Un objetivo puede ser rendir un examen, alcanzar un nivel de idioma o dominar cualquier disciplina que admita ser dividida en temas.

La unidad fundamental es el tema: una porción de contenido con una dificultad, un tiempo estimado y, opcionalmente, subtemas que lo componen. Un tema puede requerir a otro, lo cual expresa que no tiene sentido estudiarlo antes de haber comprendido a su predecesor. Estas dependencias conforman un grafo dirigido cuyo orden determina la secuencia admisible de estudio.

Cada tema puede asociarse a un método de estudio, que determina cómo se ocupa el tiempo asignado. El método pomodoro fracciona la sesión en ciclos de trabajo y descanso de duración fija, mientras que la repetición espaciada expande una única declaración en varios repasos ubicados a intervalos crecientes (un día, tres días, una semana y tres semanas). La elección del método tiene, por lo tanto, consecuencias concretas sobre el cronograma resultante.

El compilador genera como salida un documento HTML con el cronograma resuelto, el grafo de dependencias y el resumen de horas por objetivo, junto con un archivo iCal importable en cualquier aplicación de calendario. Toda la resolución ocurre dentro de la memoria del programa, sin integración con servicios externos de agenda. Su principal ventaja es permitir verificar la factibilidad de un plan antes de comenzar: si la carga horaria excede el tiempo disponible o el orden de estudio es contradictorio, el programa no compila.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Se podrán declarar uno o varios objetivos de aprendizaje, y cada uno tendrá nombre, fecha límite y, opcionalmente, una prioridad relativa expresada como porcentaje.
- (II). Se podrán declarar uno o varios temas dentro de cada objetivo, y cada uno tendrá nombre, dificultad en el rango 1 a 5 y tiempo estimado de estudio.
- (III). Los temas podrán contener otros temas de forma anidada, en cuyo caso el tiempo estimado del tema contenedor se obtiene como la suma de los tiempos de sus subtemas.
- (IV). Se podrán declarar dependencias de conocimiento entre temas mediante la construcción `requires`, admitiendo más de un predecesor por tema.
- (V). El lenguaje admitirá valores de tipo `duration`, `date`, `time`, `integer`, `percentage`, `boolean` y `string`, que podrán utilizarse como atributos y dentro de expresiones.
- (VI). Se podrán sumar(+) y restar(-) valores de tipo `duration`, así como multiplicar(*) o dividir(/) una duración por un valor entero. Toda operación entre tipos incompatibles será rechazada.
- (VII). Se proveerán operadores relacionales como `<`, `>`, `=`, `7`, `≤` y `≥`.
- (VIII). Se proveerán operaciones lógicas básicas como `AND`, `OR` y `NOT`.
- (IX). Se podrá asociar a cada tema un método de estudio, declarado como un bloque con parámetros propios según el método seleccionado.
- (X). El método de repetición espaciada admitirá una lista de intervalos, y expandirá la declaración de un único tema en tantas sesiones de repaso como intervalos se hayan indicado.
- (XI). Se podrán definir perfiles reutilizables que agrupen un método y sus parámetros, y aplicarlos a varios temas mediante la construcción `use`.
- (XII). Se podrá declarar la disponibilidad horaria semanal, asignando una cantidad de horas a cada día e indicando los días de descanso.
- (XIII). Se podrán declarar restricciones globales sobre el cronograma, tales como la cantidad máxima de temas distintos por jornada o la exigencia de finalizar el repaso de un objetivo con cierta antelación a su fecha límite.
- (XIV). Se proveerán estructuras de control de tipo `IF-THEN-ELSE` que permitan condicionar la asignación de un método según los atributos del tema o la proximidad de la fecha límite.
- (XV). Se podrán fijar sesiones en una fecha y hora determinadas, las cuales no serán reubicadas por el planificador.
- (XVI). Todo programa deberá contener un bloque de planificación que delimite el rango temporal a resolver, seleccione la estrategia de asignación y declare los artefactos de salida a generar.
- (XVII). El compilador producirá como salida un documento HTML con el cronograma resuelto, el grafo de dependencias y el resumen de horas por objetivo, junto con un archivo iCal importable en cualquier aplicación de calendario.

5. Casos de prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que declare un objetivo con un único tema, dificultad y duración.
- (II). Un programa que declare varios objetivos con distintas fechas límite.
- (III). Un programa que establezca una dependencia lineal entre tres temas.
- (IV). Un programa donde un tema dependa de dos temas anteriores.
- (V). Un programa que declare un tema compuesto por varios subtemas.
- (VI). Un programa que utilice una duración compuesta, como 2h 30m.
- (VII). Un programa que aplique el método Pomodoro a un tema.
- (VIII). Un programa que aplique repetición espaciada con varios intervalos.
- (IX). Un programa que defina un perfil y lo reutilice en dos temas.
- (X). Un programa que declare disponibilidad semanal y días de descanso.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa sintacticamente malformado.
- (II). Un programa que declare dos objetivos con el mismo nombre.
- (III). Un programa que asocie un tema a un objetivo inexistente.
- (IV). Un programa cuya carga de estudio no pueda completarse antes de la fecha límite.
- (V). Un programa con una dificultad fuera del rango 1 a 5.
- (VI). Un programa con una duración negativa o igual a cero.
- (VII). Un programa que opere entre tipos incompatibles.
- (VIII). Un programa cuyo grafo de dependencias contenga un ciclo.
- (IX). Un programa con parámetros inválidos para un método de estudio.
- (X). Un programa que ubique una sesión fija fuera de la disponibilidad declarada.

6. Ejemplos

En el **Cód. (6.1)**, se puede ver un objetivo académico con tres temas encadenados por dependencias, junto con la disponibilidad horaria y el bloque de planificación que resuelve el cronograma.

```
// Declarar un objetivo con su fecha limite:
goal TLA due 2026-12-05 priority 80%;

// Declarar los temas y su orden de estudio:
topic Automatas in TLA {
  difficulty 3;
  estimated 6h;
}
topic Gramaticas in TLA {
  requires Automatas;
  difficulty 4;
  estimated 4h 30m;
}
topic Parsing in TLA {
  requires Gramaticas;
  difficulty 5;
  estimated 3h * 4h;
  method pomodoro {
    work 25m;
    break 5m;
    cycles 4;
  }
}

// Declarar las horas disponibles por dia:
availability {
  mon, wed, fri = 3h;
  sat = 5h;
  sun = rest;
}

// Resolver el cronograma y emitir los artefactos:
plan from 2026-09-01 until deadlines {
  export calendar, summary;
}
```

Código 6.1: Objetivo academico con dependencias entre temas.

En el **Cód. (6.2)**, se declara un objetivo de aprendizaje de un idioma, ajeno al ámbito académico, con temas anidados y un perfil reutilizable basado en repetición espaciada aplicado condicionalmente. El tema Vocabulario expande su declaración en cuatro sesiones de repaso, ubicadas por el compilador a uno, tres, siete y veintiún días de la sesión inicial.

```
// Definir un perfil reutilizable con sus repeticion espaciadas
profile memorizacion {
  method spacedRepetition {
    intervals 1d, 3d, 7d, 21d;
  }
  priority 90%;
}

// Objetivo: alcanzar un nivel de idioma.
goal FrancesB2 due 2027-03-15;

// Se agrega tema de vocabulario a el obj.
topic Vocabulario in FrancesB2 {
  difficulty 5;
  estimated 40h;

  // Lo difícil se repasa; el resto se estudia una sola vez:
  if (topic.difficulty > 4) {
    use memorizacion;
  } else {
    method activeRecall { sessions 2; }
  }
}

// Tema compuesto: su duración es la suma de sus subtemas.
topic Gramatica in FrancesB2 {
  requires Vocabulario;
  topic Subjuntivo { estimated 12h; }
  topic Condicional { estimated 8h; }
}
```

```
// Restringir la carga diaria y exigir margen antes de la fecha:
constraint noMoreThan 2 topicsPer d;
constraint reviewBeforeT 7d;
plan from 2026-09-01 until deadlines {
  strategy earliestDeadlineFirst;
  export calendar, summary;
}
```

Código 6.2: Objetivo de idioma con temas anidados y repeticion espaciada.